

Coding Cheat Sheet

This reading provides a reference list of code that you'll encounter as you work with object-oriented coding in Java. Understanding these concepts will help you write and debug object-oriented Java programs. Let's explore the following Java coding concepts:

- Working with classes and objects
- Working with access and non-access modifiers
- Using encapsulation
- Using constructors

Keep this summary reading available as a reference as you progress through your course, and refer to it when you begin coding object-oriented Java programming after this course!

Working with classes and objects

Creating a class

Description	Example
Create a Car class, which serves as a blueprint for creating Car objects.	<pre>public class Car {</pre>
Define attributes of the Car class. The variables color, model, and year store the car's color, model, and year, respectively.	<pre> String color; String model; int year;</pre>
Include the method displayInfo() to print car objects.	<pre> void displayInfo() {</pre>
Print the car details to the console using the System.out.println() function.	<pre> System.out.println("Car Model: " + model); System.out.println("Car Color: " + color); System.out.println(>System.out.println("Car Year: " + year);</pre>
Close curly braces to end the Car class definition.	<pre> } }</pre>

Explanation: This example creates a class named Car and defines three attributes for the Car class: model, color, and year. The displayInfo() method prints the car details.

Creating an object

Description	Example
A Java class named <code>Main</code> with a <code>main</code> method. The <code>main</code> method is the entry point of the program.	<pre>public class Main {</pre>
The <code>main</code> method is declared using <code>public static void main(String[] args)</code> . This method is required for execution in Java programs.	<pre>public static void main(String[] args) {</pre>
Create an object of the <code>Car</code> class.	<pre>Car myCar = new Car();</pre>
Assign values to the object's attributes.	<pre>myCar.color = "Red"; myCar.model = "Toyota"; myCar.year = 2020;</pre>
Call the <code>displayInfo()</code> method to print the object details.	<pre>myCar.displayInfo();</pre>
Close curly braces to end the <code>main</code> method and class definition.	<pre> } }</pre>

Explanation: This example declares a reference variable named `myCar` of type `Car`. `new Car()` creates a new object of the `Car` class and assigns values to the object's attributes: `color`, `model` and `year`. The `displayInfo()` method prints the car details.

Working with access and non-access modifiers

Public access modifier

Description	Example
A Java statement used to define a class named <code>Car</code> , which acts as a blueprint for creating <code>Car</code> objects. The variable <code>model</code> is declared as <code>public</code> , meaning it can be accessed directly from outside the class.	<pre>public class Car {</pre>

Description	Example
A Java statement to declare a <code>String</code> variable named <code>model</code> to store the car's model name.	<code>public String model;</code>
Close curly braces to end the class definition.	<code>}</code>

Private access modifier

Description	Example
A Java statement used to define a class named <code>Car</code> , which acts as a blueprint for creating <code>Car</code> objects. The variable <code>model</code> is declared as <code>public</code> , meaning that it can be accessed directly from outside the class.	<code>public class Car {</code>
A Java statement to declare a private <code>String</code> variable named <code>color</code> to store the car's color. The <code>private</code> modifier ensures the <code>color</code> variable can be accessed and modified only within the <code>Car</code> class.	<code>private String color;</code>
Call the <code>displayColor()</code> method with the <code>private</code> access modifier. This ensures the method can be called only within the <code>Car</code> class and not from other classes.	<code>private void displayColor() {</code>
Print the car's color to the console using the <code>System.out.println()</code> function.	<code>System.out.println("Car Color: " + color);</code>
Close curly braces to end the class definition.	<code> } }</code>

Description	Example

Protected access modifier

Description	Example
A Java statement used to define a class named <code>Car</code>, which acts as a blueprint for creating <code>Car</code> objects. The variable <code>model</code> is declared as <code>public</code>, meaning that it can be accessed directly from outside the class.	<pre>public class Car {</pre>
A Java statement to declare a protected <code>int</code> variable named <code>year</code> to store the car's year. The protected modifier ensures the <code>year</code> variable is accessible within the same package (default package access) and by subclasses, even if they are in different packages.	<pre>private String year;</pre>
Call the <code>displayYear()</code> method with the protected access modifier. This ensures the method can be called within the same package and by subclasses, even if they are in different packages.	<pre>private void displayYear() {</pre>
Print the car's year to the console using the <code>System.out.println()</code> function.	<pre>System.out.println("Car Year: " + year);</pre>
Close curly braces to end the class definition.	<pre>} }</pre>

Default access modifier

Description	Example
A Java statement used to define a class named <code>Car</code>, which acts as a blueprint for creating <code>Car</code> objects.	<pre>class Car {</pre>
A Java statement to declare a <code>String</code> variable named <code>model</code> without any access modifier. If no access modifier is used, the variable is considered default. Default variables are accessible only within their own package.	<pre>String model;</pre>

Description	Example
Call the <code>displayModel()</code> method without any access modifier.	<pre>void displayModel() {</pre>
Print the car's model to the console using the <code>System.out.println()</code> function.	<pre>System.out.println("Car Model: " + model);</pre>
Close curly braces to end the class definition.	<pre>} }</pre>

Static non-access modifier

Description	Example
A Java statement used to define a class named <code>Car</code> , which acts as a blueprint for creating <code>Car</code> objects. The variable <code>model</code> is declared as <code>public</code> , meaning that it can be accessed directly from outside the class.	<pre>public class Car {</pre>
A Java statement to declare a static <code>int</code> variable named <code>numberOfCars</code> to keep track of the total number of <code>Car</code> objects created. Since it's static, its value is shared among all instances of <code>Car</code> .	<pre>static int numberOfCars = 0;</pre>
A Java statement to declare a constructor. Every time a new <code>Car</code> object is created, this constructor runs.	<pre>public Car() {</pre>
A Java statement to increment the <code>numberOfCars</code> variable that keeps track of how many cars have been instantiated.	<pre>numberOfCars++;</pre>

Description	Example
Close curly braces to end the class definition.	}
Call the <code>displayCount()</code> method without creating an instance of the <code>Car</code> class. This method can only access static variables like <code>numberOfCars</code> , not instance variables.	<pre>private void displayCount() {</pre>
Print the total number of cars to the console using the <code>System.out.println()</code> function.	<pre>System.out.println("Total Cars: " + numberOfCars);</pre>
Close curly braces to end the class definition.	<pre>} }</pre>

Final non-access modifier

Description	Example
A Java statement used to define a final class named <code>Vehicle</code> , which acts as a blueprint for creating <code>Car</code> objects. The final class cannot be extended (inherited) by any other class. This means no subclasses can be created from <code>Vehicle</code> .	<pre>public final class Vehicle {</pre>
A Java statement to declare a final <code>int</code> variable named <code>maxSpeed</code> with the value 120. The final variable is a constant, meaning that its value cannot be changed once it is assigned. Trying to modify <code>maxSpeed</code> later in the code will cause a compilation error.	<pre>final int maxSpeed = 120;</pre>
A Java statement to declare a final method named <code>displayMaxSpeed()</code> . The final method cannot be overridden by subclasses. This ensures the behavior of <code>displayMaxSpeed</code> remains the same in all instances.	<pre>final void displayMaxSpeed() {</pre>

Description	Example
Print the maximum car speed to the console using the <code>System.out.println()</code> function.	<pre>System.out.println("Max Speed: " + maxSpeed);</pre>
Close curly braces to end the class definition.	<pre>} }</pre>

Abstract non-access modifier

Description	Example
A Java statement used to define an abstract class named <code>Shape</code> . This is an abstract class, meaning that it cannot be instantiated (you cannot create <code>Shape</code> objects directly). It works as a blueprint from which other classes can inherit.	<pre>public abstract class Shape {</pre>
A Java statement used to define a final class named <code>Vehicle</code> , which acts as a blueprint for creating <code>Car</code> objects. The final class cannot be extended (inherited) by any other class. This means no subclasses can be created from <code>Vehicle</code> .	<pre>abstract void draw();</pre>
Close curly braces to end the class definition.	<pre>}</pre>
A Java statement to describe <code>Circle</code> that extends the <code>Shape</code> class and provides an implementation of the <code>draw()</code> method.	<pre>public class Circle extends Shape {</pre>
A Java annotation to tell the compiler the <code>draw()</code> method in <code>Circle</code> is an override of the abstract method in <code>Shape</code> .	<pre>@Override</pre>

Description	Example
A Java statement saying the draw method is now fully implemented.	<pre>void draw()</pre>
Print the string Drawing Circle to the console using the System.out.println() function.	<pre>System.out.println("Drawing Circle");</pre>
Close curly braces to end the class definition.	<pre> } }</pre>

Using encapsulation

Creating an encapsulated class

Description	Example
Create the Person class, which serves as a blueprint for creating Person objects.	<pre>class Person {</pre>
Create private attributes name and age to store the person's name and age. The name and age attributes cannot be accesse directly from outside the class.	<pre> private String name; private int age;</pre>
Use the Java constructor to initialize the name and age variables when a Person object is created.	<pre> public Person(String name, int age) {</pre>
The keyword this refers to the current object's instance variables. It differentiates instance variables from method parameters.	<pre> this.name = name; this.age = age;</pre>

Description	Example
Close curly braces to end the class definition.	}
Use the Java public method (Getter) to obtain read access to private variables.	public String getName() {
getName() returns the value of name.	return name;
Close curly braces to end the class definition.	}
Use the Java public method (Setter) to obtain write access to private variables.	public void setName(String name) {
setName() updates name.	this.name = name;
Use the Java public method (Getter) to obtain read access to private variables.	public int getAge() {
getAge() returns the value of age.	return age;

Description	Example
Close curly braces to end the class definition.	}
Use the Java public method (Setter) to obtain write access to private variables.	public void setAge(int age) {
Use the Java if statement to ensure age is not negative before assigning.	if (age >= 0) {
Update the age variable.	this.age = age;
Use the Java else statement to specify what to do when the age is negative.	} else {
Print the string Age cannot be negative to the console using the System.out.println() function.	System.out.println("Age cannot be negative.");
Close curly braces to end the class definition.	} }

Explanation: This example creates a Person class in which the name and age attributes are declared as private, meaning they cannot be accessed directly from outside the Person class. The constructor Person(String name, int age) initializes the attributes when a new object of the class is created. getName() and getAge() are getter

methods that allow other classes to read the values of `name` and `age`. `setName(String name)` and `setAge(int age)` are setter methods that allow other classes to modify the values of `name` and `age`. The setter for `age` includes validation to ensure `age` cannot be set to a negative number.

Using an encapsulated class

Description	Example
A Java class named <code>Main</code> with a <code>main</code> method. The <code>main</code> method is the entry point of the program.	<pre>public class Main {</pre>
The <code>main</code> method is declared using <code>public static void main(String[] args)</code> . This method is required for execution in Java programs.	<pre>public static void main(String[] args) {</pre>
Create a new instance of the <code>Person</code> class. Assign the value "Alice" to the <code>name</code> attribute and the value "30" to the <code>age</code> attribute.	<pre>Person person = new Person("Alice", 30);</pre>
Use the <code>getName()</code> getter to obtain and print the value of the <code>name</code> attribute.	<pre>System.out.println("Name: " + person.getName());</pre>
Use the <code>getAge()</code> getter to obtain and print the value of the <code>age</code> attribute.	<pre>System.out.println("Age: " + person.getAge());</pre>
Use the <code>setName()</code> setter to assign the value of <code>name</code> attribute to "Bob" and <code>age</code> attribute to "25".	<pre>person.setName("Bob"); person.setAge(25);</pre>
Use the <code>getName()</code> getter to obtain and print the updated value of the <code>name</code> attribute.	<pre>System.out.println("Updated Name: " + person.getName());</pre>

Description	Example
The <code>setAge(-5)</code> call attempts to set an invalid age. Since <code>setAge()</code> has validation logic, it will print "Age cannot be negative."	<pre>System.out.println("Updated Age: " + person.getAge());</pre>
Close curly braces to end the class definition.	<pre> } }</pre>

Explanation: This example creates an instance of the `Person` class with the name "Alice" and age "30". We call the `getName()` and `getAge()` getter methods to print the values. We then update the name and age attributes using the `setName()` and `setAge()` setter methods. When we attempt to set a negative age with `setAge(-5)`, it prints an error message because of validation included in the setter method.

Using constructors

Creating a default constructor

Description	Example
A Java statement used to define a class named <code>Dog</code> , which acts as a blueprint for creating <code>Dog</code> objects.	<pre>class Dog {</pre>
A Java statement to declare a <code>String</code> variable named <code>name</code> without any access modifier. If no access modifier is used, the variable is considered default. Default variables are accessible only within their own package.	<pre> String name;</pre>
This is the default constructor. It takes no arguments.	<pre> Dog() {</pre>
The default constructor initializes the <code>name</code> variable with the value "Unknown". This ensures every new <code>Dog</code> object always has a name, even if the user doesn't provide one.	<pre> name = "Unknown";</pre>
Close curly braces to end the class definition.	<pre> } }</pre>

Description	Example
Call the <code>display()</code> method without any access modifier.	<pre>void display() {</pre>
Print the dog's name to the console using the <code>System.out.println()</code> function. Since <code>name</code> was initialized in the constructor, it always has a value.	<pre>System.out.println("Dog's name: " + name);</pre>
Close curly braces to end the class definition.	<pre>} }</pre>
A Java class named <code>Main</code> with a <code>main</code> method. The <code>main</code> method is the entry point of the program.	<pre>public class Main {</pre>
The <code>main</code> method is declared using <code>public static void main(String[] args)</code> . This method is required for execution in Java programs.	<pre>public static void main(String[] args) {</pre>
Create an instance of the <code>Dog</code> class using the default constructor. The <code>name</code> variable is automatically set to "Unknown".	<pre>Dog myDog = new Dog();</pre>
Call the <code>display()</code> method to print the dog's name.	<pre>myDog.display();</pre>
Close curly braces to end the class definition.	<pre>} }</pre>

Description	Example

Explanation: This example creates an instance of the `Dog` class with a default constructor that initializes the `name` attribute to "Unknown". When we create the instance, the default constructor is invoked automatically.

Creating a parameterized constructor

Description	Example
A Java statement used to define a class named <code>Dog</code> , which acts as a blueprint for creating <code>Dog</code> objects.	<pre>class Dog {</pre>
A Java statement to declare a <code>String</code> variable named <code>name</code> without any access modifier. If no access modifier is used, the variable is considered default. Default variables are accessible only within their own package.	<pre>String name;</pre>
This is the parameterized constructor that takes one argument <code>dogName</code> .	<pre>Dog(String dogName) {</pre>
When the <code>Dog</code> object is created, the provided <code>dogName</code> value is assigned to the <code>name</code> variable. Parameterized constructors let you assign a unique name to each <code>Dog</code> object when it is created.	<pre>name = dogName;</pre>
Close curly braces to end the class definition.	<pre>}</pre>
Call the <code>display()</code> method without any access modifier.	<pre>void display() {</pre>

Description	Example
Print the dog's name to the console using the <code>System.out.println()</code> function. Since <code>name</code> was initialized in the constructor, it always has a value.	<pre>System.out.println("Dog's name: " + name);</pre>
Close curly braces to end the class definition.	<pre>} }</pre>
A Java class named <code>Main</code> with a <code>main</code> method. The <code>main</code> method is the entry point of the program.	<pre>public class Main {</pre>
The <code>main</code> method is declared using <code>public static void main(String[] args)</code> . This method is required for execution in Java programs.	<pre>public static void main(String[] args) {</pre>
Create an instance of the <code>Dog</code> class. "Buddy" is passed as an argument to the constructor, setting name to "Buddy".	<pre>Dog myDog = new Dog("Buddy");</pre>
Call the <code>display()</code> method to print the dog's name.	<pre>myDog.display();</pre>
Close curly braces to end the class definition.	<pre>} }</pre>

Explanation: This example creates an instance of the `Dog` class with a parameterized constructor that takes a string parameter `dogName`. When we create a `Dog` instance with the name "Buddy", the constructor initializes the `name` attribute with that value.

Creating a no-arg constructor

Description	Example
A Java statement used to define a class named <code>Car</code> , which acts as a blueprint for creating <code>Car</code> objects.	<pre>class Car {</pre>
A Java statement to declare a <code>String</code> variable named <code>model</code> and an <code>int</code> variable named <code>year</code> without any access modifier. If no access modifier is used, the variable is considered default. Default variables are accessible only within their own package.	<pre>String model; int year;</pre>
This is a no-argument constructor that takes no parameters.	<pre>Car() {</pre>
When the <code>Car</code> object is created, it automatically assigns the value "Default Model" to <code>model</code> and 2020 to <code>year</code> .	<pre>model = "Default Model"; year = 2020;</pre>
Close curly braces to end the class definition.	<pre>}</pre>
Call the <code>display()</code> method without any access modifier.	<pre>void display() {</pre>
Print the car's model and year to the console using the <code>System.out.println()</code> function.	<pre>System.out.println("Car Model: " + model + ", Year: " + year);</pre>
Close curly braces to end the class definition.	<pre> } }</pre>

Description	Example
A Java class named <code>Main</code> with a <code>main</code> method. The <code>main</code> method is the entry point of the program.	<pre>public class Main {</pre>
The <code>main</code> method is declared using <code>public static void main(String[] args)</code> . This method is required for execution in Java programs.	<pre>public static void main(String[] args) {</pre>
Create an instance of the <code>Car</code> class. The no-argument constructor is called, setting <code>model</code> = "Default Model" and <code>year</code> = 2020.	<pre>Car myCar = new Car();</pre>
Call the <code>display()</code> method to print the model and year of the car.	<pre>myCar.display();</pre>
Close curly braces to end the class definition.	<pre> } }</pre>

Explanation: This example creates an instance of the `Car` class with two attributes `model` and `year`. The `Car()` constructor initializes the `model` to "Default Model" and `year` to 2020. When we create an instance of the `Car` class with `new Car()`, the no-arg constructor is called automatically, and the default values are assigned to the attributes. The `display()` method prints the `model` and `year` of the car.

Constructor overloading

Description	Example
A Java statement used to define a class named <code>Dog</code> , which acts as a blueprint for creating <code>Dog</code> objects.	<pre>class Dog {</pre>
A Java statement to declare a <code>String</code> variable named <code>name</code> and an <code>int</code> variable named <code>age</code> without any access modifier. If no access modifier is used, the variable is considered default. Default variables are accessible only within their own package.	<pre>String name; int age;</pre>

Description	Example
This is the default constructor. It takes no arguments.	<pre>Dog() {</pre>
The default constructor initializes the <code>name</code> variable with the value "Unknown" and <code>age</code> variable with the value 0. This ensures every new <code>Dog</code> object always has a name and age, even if the user doesn't provide one.	<pre> name = "Unknown"; age = 0;</pre>
Close curly braces to end the class definition.	<pre>}</pre>
This is the parameterized constructor that takes one argument <code>dogName</code> .	<pre>Dog(String dogName) {</pre>
When the <code>Dog</code> object is created, the provided <code>dogName</code> value is assigned to <code>name</code> while keeping the <code>age</code> as 0 by default. Parameterized constructors let you assign a unique name to each <code>Dog</code> object when it is created.	<pre> name = dogName; age = 0;</pre>
Close curly braces to end the class definition.	<pre>}</pre>
This is the parameterized constructor that takes two arguments <code>dogName</code> and <code>dogAge</code> .	<pre>Dog(String dogName, int dogAge) {</pre>

Description	Example
When the Dog object is created, the constructor allows the user to specify both name and age.	<pre>name = dogName; age = dogAge;</pre>
Close curly braces to end the class definition.	<pre>}</pre>
Call the display() method without any access modifier.	<pre>void display() {</pre>
Print the dog's name and age to the console using the System.out.println() function. Since name and age were initialized in the constructor, they always have a value.	<pre>System.out.println("Dog's name: " + name + ", Age: " + age);</pre>
Close curly braces to end the class definition.	<pre> } }</pre>
A Java class named Main with a main method. The main method is the entry point of the program.	<pre>public class Main {</pre>
The main method is declared using public static void main(String[] args). This method is required for execution in Java programs.	<pre>public static void main(String[] args) {</pre>
Create the dog1 object using the default constructor Dog(). So, name = "Unknown" and age = 0.	<pre>Dog dog1 = new Dog();</pre>

Description	Example
Create the <code>dog2</code> object using the one-parameter constructor <code>Dog("Charlie")</code> . So, <code>name = "Charlie"</code> and <code>age = 0</code> (default).	<pre>Dog dog2 = new Dog();</pre>
Create the <code>dog3</code> object using the two-parameter constructor <code>Dog("Max", 5)</code> . So, <code>name = "Max"</code> and <code>age = 5</code> .	<pre>Dog dog3 = new Dog();</pre>
Call the <code>display()</code> method on each object to print their details.	<pre>dog1.display(); dog2.display(); dog3.display();</pre>
Close curly braces to end the class definition.	<pre>} }</pre>

Explanation: This example has three constructors of the `Dog` class. Depending on the number of parameters provided when creating an object, the corresponding constructor is called.

Author(s)

Ramanujam Srinivasan
Lavanya Thiruvalli Sunderarajan



Skills Network